Software Test Plan (Validation Specification v1)

Project: Railway Reservation System (RRS) — Team 5, UE24CS341A

Version: 1.0

Authors: Ridhima Jain, S. S. Adhithya Sriram, Saathvik Shenoy, Padubidri, and Samhitha S. (QA: all members)

Date: 28-09-2026

1. Introduction

Purpose: This document defines the test plan for RRS v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing the four functional modules and their supporting NFR/security requirements.

Scope: Testing covers registration/login/RBAC, inventory management, graph-based search with fare/eco-scoring, and booking/waitlist/cancellation. Excludes real payment settlement, GPS tracking, and native mobile apps (out of scope per SRS §2.3).

References: RRS SRS v1.1, RRS Software Architecture & Design Specification v1.0, PES Jackfruit – Software Engineering Project Guidelines 2026.

Definitions: PNR, RBAC, TUI, Waitlist, and Eco-score (see SRS Appendix A).

2. Test Items

- Auth & RBAC module
- Inventory & Schedule module (Admin)
- Search Engine, Fare & Eco-Optimization module
- Booking & Queue Management module
- Mock Payment interface
- ncurses TUI

3. Features to be Tested

Features mapped to SRS requirement IDs:

Requirement ID	**Feature**
FR-001 – FR-006	Registration, login, session, RBAC
FR-007 – FR-011	Train/route/capacity management
FR-012 – FR-017	Route search, fare, and eco-score
FR-018 – FR-024	Booking, PNR, waitlist, cancellation
NFR-001 – NFR-009	Performance, availability, usability, reliability, maintainability, testability, scalability, integrity
SEC-001 – SEC-006	Password storage, RBAC enforcement, input validation, session expiry, payment data handling, audit logging

4. Features Not to be Tested

- Real-money payment settlement / bank integration (mocked, out of scope)
- Live GPS train tracking
- Native mobile applications (only TUI and optional web wrapper are in scope)
- Multi-tenant deployment across multiple railway operators

5. Test Approach / Strategy

Levels

- Unit tests (GTest/Unity, module-level; each owner tests their own module)
- Integration tests (Search → Booking → Waitlist; Auth → all modules via RBAC)
- System tests (end-to-end TUI walkthrough)
- Acceptance tests (instructor/TA walkthrough of the demo scenarios)

Types

- Functional testing of all FR-IDs
- Negative/boundary testing (invalid input, capacity conflicts, payment failure)
- Performance testing (NFR-001, NFR-002 response-time targets)
- Static analysis and memory testing (cppcheck, Valgrind)

Entry Criteria: stable build on main, seed data (stations/trains) loaded, CI green.

Exit Criteria: all high-priority test cases executed, 0 critical defects open, ≥ 70% unit coverage per module, and NFR-001/NFR-002 targets met.

5.1 Security Validation

Security objectives: SO-1 credential/data confidentiality; SO-2 restrict privileged actions; SO-3 booking integrity; SO-4 auditability.

ID	Objective	Test approach
SEC-001	SO-1	Inspect the stored user file; confirm salted hashes only, and salts differ per user.
SEC-002	SO-2	Passenger session calls an admin-only function; expect ERR_FORBIDDEN.
SEC-003	SO-3	Fuzz all input fields with overlong strings, special characters, and negative numbers.
SEC-004	SO-2	Reuse an expired/invalid session token; expect ERR_SESSION_EXPIRED.
SEC-005	SO-1	Inspect booking records; confirm no payment details are persisted.
SEC-006	SO-4	Trigger 3 failed logins and an admin action; confirm audit-log entries.

6. Test Environment

Hardware: standard developer laptops (Linux/WSL) for build and unit testing; a single Ubuntu evaluation machine for the demo.

Software: GCC/G++, ncurses, Make/CMake, gtest/Unity, cppcheck, Valgrind, Docker.

Tools: GitHub Actions (CI), GitHub Projects (backlog/board), gcov/lcov (coverage).

Test Data: synthetic seeded stations, trains, and passenger accounts (no real personal or payment data, per SRS §10).

7. Test Schedule

Milestone	Date
Test case design	02-Oct-2026
Unit tests for Sprint 1–2 modules	11-Oct-2026
Integration test execution starts.	12-Oct-2026
System / security / performance testing	19–25 Oct 2026
Final validation report	25-Oct-2026

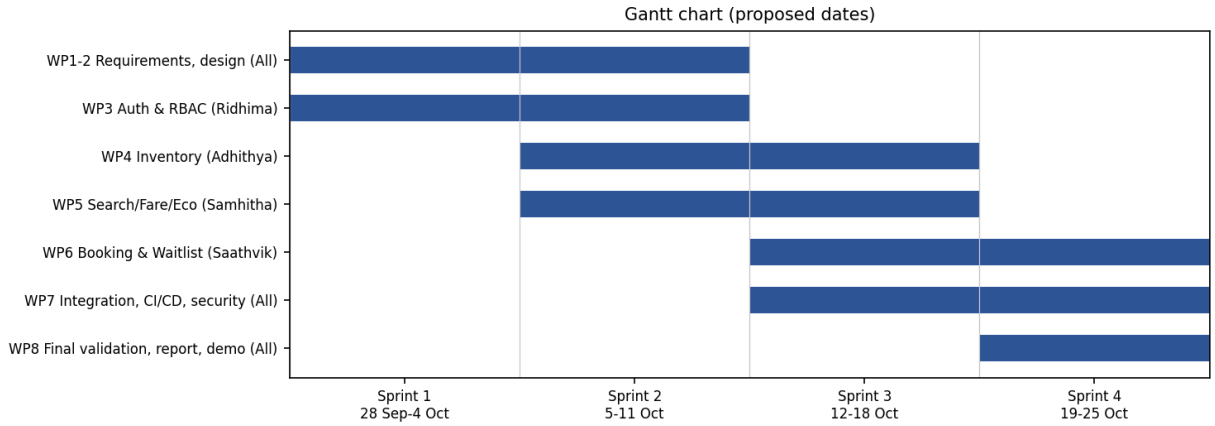


Figure 1: Sprint schedule (Gantt) across the four-week plan

8. Test Deliverables

- Test Plan (this document)
- Test cases
- Unit test report and coverage report
- CI/CD pipeline evidence
- Security validation evidence
- Test execution logs and defect reports

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Samhitha S	Prepare a plan, coordinate execution, and own search module tests.
Test Engineer	Ridhima Jain	Auth/RBAC test design & execution
Test Engineer	S S Adhithya Sriram	Inventory module test design & execution
Test Engineer	Saathvik Shenoy Padubidri	Booking/waitlist test design & execution

Product Owner / Evaluator	Course Instructor	Approve test results, sign off on readiness

10. Risks and Mitigation

Risk	Mitigation
ncurses unavailable on evaluator's machine	Plain CLI fallback build target
Sprint 3 schedule slip (booking/waitlist complexity)	Start data structures in Sprint 2, integrate early.
Test environment inconsistency across four laptops	Standardize on a shared Docker dev image.

11. Assumptions & Dependencies

- Train, station, and route master data will be seeded by the team, not sourced from a live external database
- ncurses (or an equivalent) is available on the evaluation machine; fallback is the API + web wrapper
- GitHub Actions (or an equivalent CI tool) remains available throughout the project

12. Suspension & Resumption Criteria

Suspend testing if the build breaks on main or seed data is corrupted such that more than 30% of test cases are blocked.

Resume testing when the blocking defect is fixed and merged and the environment is verified stable.

13. Test Case Management & Traceability

The Requirement Traceability Matrix (RTM) maps SRS requirement IDs to test case IDs, extending SRS Appendix C with a System Test ID column. Example:

Req ID	Architecture ref	Design ref	Code/file ref	Test Case ID	System Test ID
FR-001	Auth & RBAC	Class: User	auth_service.*	UT-01	ST-AUTH-01
FR-012	Search/Fare/Eco	Sequence: Search Trains	search_service.*	IT-01	ST-SEARCH-01
FR-018	Booking & Waitlist	Sequence: Book Ticket	booking_service.*	IT-03	ST-BOOK-01
SEC-002	Facade / RBAC	RBAC design	middleware/auth.*	ST-01	ST-SEC-02

13.1 Detailed Test Cases

Test Case ID	Module	Type	Description	Precondition	Steps	Test Data	Expected Result	Req ID

UT-01	Search/Fare	Unit	Fare calculation for a direct route, no concession	Graph seeded with station distances	Call fare_calculate()	500 km, Sleeper, no concession	Matches hand-computed fare	FR-014
UT-02	Search/Fare	Unit	Fare calculation with senior concession	Graph seeded	Call fare_calculate() with concession flag	500 km, Sleeper, senior	Discounted fare returned	FR-014
UT-03	Search	Unit	Reject invalid station code	Graph loaded	Search with a bad station code	Origin='XXX'	ERR_INVALID_INPUT	FR-012
IT-01	Search	Integration	Direct route search returns all matching trains	Seeded network A–B with 3 trains	search_routes('A','B',date)	A → B	All 3 direct trains listed, ranked	FR-012
IT-02	Search	Integration	Connected (multi-leg) route search when no direct train exists	No direct A–C train; B connects them	search_routes('A','C',date)	A → C	Multi-leg route via B returned	FR-013
IT-03	Booking	Integration	Booking is not created when mock payment fails	Valid search result selected	booking_create() with payment mocked to fail	Payment=FAIL	No PNR created; ERR_PAYMENT_FAILED	FR-019
IT-04	Booking	Integration	Waitlist enqueue and automatic promotion on cancellation	Train/class at full capacity	Book 1 more (waitlisted), then cancel 1 confirmed booking	Capacity=N, bookings=N+1	New booking WAITLISTED, then promoted to CONFIRMED after cancellation	FR-020, FR-022
ST-01	Auth/RBAC	System	Passenger session cannot call an admin-only function	Logged in as passenger	Call train_create() with passenger session	role=PASSENGER	ERR_FORBIDDEN returned	FR-004, SEC-002
ST-02	Search	System / Performance	Search responds within the NFR-001 time budget	200 stations, 500 trains seeded	Run 100 concurrent search queries	Random station pairs	≥95% of queries return within 2 seconds	NFR-001

ST-03	Booking	System	Duplicate booking is not created on payment-timeout retry	Booking request sent once, times out	Retry the identical booking request	Same idempotency key	Exactly one booking exists	NFR-005, FR-024
ST-04	Auth	Security	Passwords are never stored in plaintext	User registered	Inspect the persisted user record	N/A	Only a salted hash is present; salts differ per user	SEC-001
ST-05	Booking	Security	No seat is double-booked under concurrent requests	Class has exactly 1 seat left	Fire 50 concurrent booking requests for that seat	Concurrent load=50	Exactly 1 CONFIRMED booking; rest WAITLISTED	NFR-009

14. Test Metrics & Reporting

- % of test cases executed and % passed/failed
- Defect density and defect aging
- Requirement coverage (RTM completeness)
- Unit test line/branch coverage per module

Reports: sprint-end execution status, final Test Summary Report.

15. Approvals

Role	**Name**	**Signature / Date**
QA Lead	Samhitha S	
Dev Lead		
Course Coordinator		

Test Plan v1.1 Revision: Execution and Results (13 November 2026)

Version 1.0 planned the testing; this revision records what changed in the test cases during the sprints, how the plan was executed and the results. Sections 1 to 12 of v1.0 still apply. Full reports are in the repository's `docs/` folder (Unit Test, System Validation, Code Coverage and Branch Coverage reports).

Revision History

Date	Change	Version
28.09.2026	Initial test plan	1.0
13.11.2026	Test cases updated after Sprints 1-3; execution results, coverage and completed RTM added	1.1

Execution summary (run on main, 12.11.2026)

Item	Result
Automated test cases (GoogleTest, 12 binaries, run by CTest and in CI)	266 executed, 266 passed, 0 failed
System tests through the ncurses TUI (ST-TUI-01 to 09)	9 executed, 8 passed, 1 found defect D-01 (medium, open)
Line / branch / function coverage of the engine (gcov + gcovr)	95.0% / 86.1% / 99.2%; every module at least 87% lines
Static analysis and memory checks	cppcheck clean; ThreadSanitizer, AddressSanitizer and UndefinedBehaviorSanitizer: 0 findings
Requirements validated	39 of 39 (FR-001 to FR-024, NFR-001 to NFR-009, SEC-001 to SEC-006)

Exit criteria (section 5)

Criterion	Status
All high-priority test cases executed	Met
0 critical defects open	Met (D-01 is medium)
At least 70% unit coverage per module	Met (87% to 100%)
NFR-001 and NFR-002 targets met	Met (search p95 46.5 ms vs 2 s; booking p95 0.003 ms vs 1 s)

13.1 Detailed Test Cases: updated and executed

Test Case ID	Req ID	Expected result	Automated test	Status	Change since v1.0
UT-01	FR-014	Rs 320 (20 + 500 x 0.60)	`FareTest.DirectSleeperNoConcessionMatchesHandComputedFare`	Pass	fare_calculate takes class and a concession enum
UT-02	FR-014	Rs 200 (40% off the distance charge)	`FareTest.SeniorConcessionDiscountsTheDistanceCharge`	Pass	concession applies to the distance charge only
UT-03	FR-012	ERR_INVALID_INPUT	`SearchInputTest.UnknownStationIsInvalidInput`	Pass	unchanged

Test Case ID	Req ID	Expected result	Automated test	Status	Change since v1.0
IT-01	FR-012	all 3 trains, fastest first	`SearchTest.DirectSearchReturnsAllMatchingTrainsRanked`	Pass	unchanged
IT-02	FR-013	route via B	`ConnectionTest.MultiLegRouteViaBIsReturnedWhenNoDirectTrain`	Pass	transfer must allow the minimum change time
IT-03	FR-019	no PNR, ERR_PAYMENT_FAILED	`BookingTest.PaymentFailureRejectsBookingAndCreatesNoPNR`	Pass	unchanged
IT-04	FR-020, FR-022	WAITLISTED, then CONFIRMED	`BookingTest.EnqueuesWaitlistWhenClassFullAndAutoPromotesOnCancel`	Pass	unchanged
ST-01	FR-004, SEC-002	ERR_FORBIDDEN, attempt audited	`SecurityValidation.Sec002PassengerCannotCallAdminFunctions`	Pass	denied attempt now written to the audit log
ST-02	NFR-001	95% within 2 s; measured p95 9 ms	`SearchPerformanceTest.DefaultSearchMeetsNfr001`	Pass	100 queries run sequentially
ST-03	NFR-005, FR-024	one booking, same PNR, charged once	`PaymentRetryTest.RetryAfterSuccessfulChargeReturnsSamePnrWithoutSecondCharge`	Pass	timeout-before-charge and repeated-timeout cases added
ST-04	SEC-001	PBKDF2 salted hashes, different salts	`SecurityValidation.Sec001PasswordsAreStoredAsSaltedHashesOnly`	Pass	in-memory record inspected (no user file yet)
ST-05	NFR-009	exactly 1 CONFIRMED of 50	`BookingTest.NFR009NoDoubleBookingUnder50ConcurrentRequestsForLastSeat`	Pass	same-session test added; ThreadSanitizer clean

Cases added during the sprints (TC-01 onward) are listed in `RRS/04_IMPLEMENTATION_TESTING/test-cases.xlsx`.

System tests through the TUI (added in Sprint 3)

System Test ID	Test	Req ID	Result
ST-TUI-01	Register a passenger from the TUI	FR-001	Pass
ST-TUI-02	Wrong password is rejected, then login works	FR-003, SEC-006	Pass
ST-TUI-03	Search, sort and price a journey	FR-012, FR-014, FR-015, FR-016	Pass
ST-TUI-04	Book and pay	FR-014, FR-018, NFR-004	Pass
ST-TUI-05	View status and cancel	FR-021, FR-023	Pass
ST-TUI-06	Admin adds a train	FR-004, FR-007, FR-010, SEC-006	Pass
ST-TUI-07	Waitlist and automatic promotion	FR-020, FR-021, FR-022, FR-023	Pass
ST-TUI-08	Admin retires a train	FR-009	Pass
ST-TUI-09	Journey on a train without that class (defect D-01)	FR-012, FR-016	Fail (D-01)

13. Requirement Traceability Matrix (completed, with System Test ID)

Req ID	Architecture ref	Design ref	Code / file ref	Test Case ID	System Test ID	Result
FR-001	Auth & RBAC	§4.3 auth_register; §4.6 User	`auth/auth.cpp, auth/password.cpp`	TC-01	ST-TUI-01	Pass
FR-002	Auth & RBAC	§4.3 auth_register (ERR_DUPLICATE)	`auth/auth.cpp`	TC-02	-	Pass
FR-003	Auth & RBAC	§4.3 auth_login; §4.6 Session	`auth/auth.cpp, auth/session.cpp`	TC-03	ST-TUI-02	Pass
FR-004	Auth & RBAC; Facade	§3.9 RBAC (SEC-002)	`auth/rbac.cpp, inventory/inventory.c`	ST-01	ST-TUI-06	Pass
FR-005	Auth & RBAC	§4.6 User	`auth/auth.cpp (auth_profile_get/update)`	TC-04	-	Pass
FR-006	Auth & RBAC	§4.6 Session	`auth/session.cpp, auth/auth.cpp (auth_logout)`	TC-05	-	Pass
FR-007	Inventory & Schedule	§4.3 train_create; §4.6 Train	`inventory/inventory.c`	TC-06	ST-TUI-06	Pass
FR-008	Inventory & Schedule	§4.3 train_update	`inventory/inventory.c`	TC-07	-	Pass
FR-009	Inventory & Schedule	§4.3 train_retire	`inventory/inventory.c`	TC-08	ST-TUI-08	Pass
FR-010	Inventory & Schedule; Search, Fare & Eco	§4.6 Route/Edge	`inventory/inventory.c, search/graph.cpp`	TC-09	ST-TUI-06	Pass
FR-011	Inventory & Schedule	§4.4 ERR_CAPACITY_CONFLICT	`inventory/inventory.c`	TC-10	-	Pass
FR-012	Search, Fare & Eco	Fig. 4 Search; §4.3 search_routes	`search/search.cpp, search/graph.cpp`	UT-03, IT-01	ST-TUI-03, ST-TUI-09	Pass*
FR-013	Search, Fare & Eco	Fig. 4 Search; §4.3 search_routes	`search/search.cpp`	IT-02	-	Pass
FR-014	Search, Fare & Eco	§4.3 fare_calculate	`search/fare_eco.cpp`	UT-01, UT-02	ST-TUI-03, ST-TUI-04	Pass
FR-015	Search, Fare & Eco	§4.3 eco_score	`search/fare_eco.cpp`	TC-11	ST-TUI-03	Pass
FR-016	Search, Fare & Eco	Fig. 4 Search (search_rank added, see SAD v1.1)	`search/fare_eco.cpp, tui/main_tui.c`	TC-12	ST-TUI-03	Pass
FR-017	Search, Fare & Eco	Fig. 4 Search	`search/search.cpp`	ST-02, TC-13	-	Pass
FR-018	Booking & Waitlist; Mock payment	Fig. 2 Book Ticket; §4.3 booking_create	`booking/booking.c, facade/facade.cpp`	TC-14	ST-TUI-04, ST-TUI-07	Pass
FR-019	Booking & Waitlist; Mock payment	Fig. 2 Book Ticket; Fig. 6 activity	`booking/booking.c`	IT-03	-	Pass
FR-020	Booking & Waitlist	Fig. 2 Book Ticket; §4.6 Waitlist	`booking/booking.c, booking/waitlist.c`	IT-04	ST-TUI-07	Pass
FR-021	Booking & Waitlist	Fig. 3 Cancel; §4.3 booking_cancel	`booking/booking.c`	TC-15	ST-TUI-05, ST-TUI-07	Pass
FR-022	Booking & Waitlist	Fig. 3 Cancel; §4.3 waitlist_promote	`booking/booking.c, booking/waitlist.c`	IT-04	ST-TUI-07	Pass
FR-023	Booking & Waitlist; TUI	§4.5 UX (status visible)	`booking/booking.c, tui/main_tui.c`	TC-16	ST-TUI-05, ST-TUI-07	Pass
FR-024	Booking & Waitlist	§4.3 idempotency key	`booking/booking.c, facade/facade.cpp`	ST-03	-	Pass
NFR-001	Search, Fare & Eco	§3.9 DoS: bounded search time	`search/search.cpp`	ST-02	-	Pass

Req ID	Architecture ref	Design ref	Code / file ref	Test Case ID	System Test ID	Result
NFR-002	Booking & Waitlist	Fig. 2 Book Ticket	`booking/booking.c`	TC-17	-	Pass
NFR-003	Facade; all modules	§4.4 error handling	`facade/facade.cpp`	TC-18	-	Pass
NFR-004	TUI	§4.5 UX (6 screens)	`tui/main_tui.c`	Code review	ST-TUI-03, ST-TUI-04	Pass
NFR-005	Booking & Waitlist; Facade	§4.3 idempotency key	`booking/booking.c, facade/facade.cpp`	ST-03	-	Pass
NFR-006	All (layered + facade)	§3.5 pattern; §4.1	`CMakeLists.txt, MODULES.md`	Code review	-	Pass
NFR-007	All	§4.1 testable without TUI	`include/*.h, tests/`	Code review	-	Pass
NFR-008	Search, Fare & Eco; Booking & Waitlist	§4.6; §4.9 adjacency list	`search/graph.cpp, inventory/inventory.c, booking/waitlist.c`	TC-19	-	Pass
NFR-009	Booking & Waitlist	§4.1 centralised booking state	`booking/booking.c (mutex)`	ST-05	-	Pass
SEC-001	Auth & RBAC	§3.9 Information disclosure	`auth/password.cpp`	ST-04	-	Pass
SEC-002	Facade / RBAC	§3.9 Elevation of privilege	`auth/rbac.cpp, inventory/inventory.c`	ST-01	-	Pass
SEC-003	Facade / input validation	§3.9 Tampering	`all modules (validation on every call)`	TC-20	-	Pass
SEC-004	Auth & RBAC	§3.9 Spoofing	`auth/session.cpp`	TC-21	-	Pass
SEC-005	Booking & Waitlist; Mock payment	§3.9 Information disclosure	`booking/booking.c`	TC-22	-	Pass
SEC-006	Audit logger	§3.9 Repudiation; §4.4	`audit/audit.cpp, auth/rbac.cpp`	TC-23	ST-TUI-02, ST-TUI-06	Pass

Defects

ID	Description	Severity	Status
D-01	Search lists a train in a class that has 0 seats on it; booking it fails with 'Invalid input' (nothing booked or charged)	Medium	Open
D-02	Inventory admin actions were not audited (SEC-006)	Medium	Fixed (#72)
D-03	Waitlist capped at 4,096 entries (NFR-008 needs 10,000)	Medium	Fixed (#72)
D-04	Data race on a shared session in booking (NFR-009)	High	Fixed (#70)
D-05	Fare supplied by the caller of booking_create (SEC-003)	Medium	Fixed (#77)